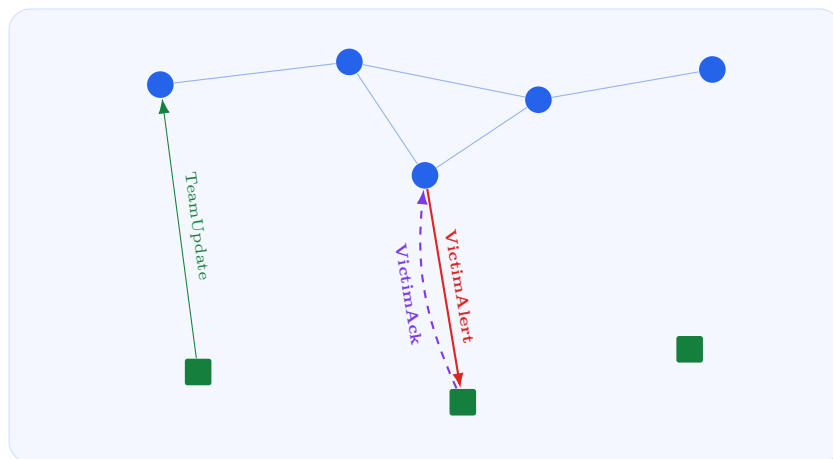


ECHOSAR-Net

Guia Ilustrado dos Logs de Simulação



Como ler, entender e explicar cada linha de log da simulação

Simulação OMNeT++ 6.2.0 + INET 4.5.4 — Dissertação PPGCAP/UDESC

Sumário

1	Por que este guia existe	2
1.1	Como gerar um log legível	2
2	Quem fala com quem: a arquitetura de mensagens	3
2.1	Diagrama de sequência do fluxo completo	3
3	Anatomia de cada mensagem	4
3.1	[DRONE] TeamUpdate — beacon da equipe	4
3.2	[DRONE] DroneStatus — resposta do drone	4
3.3	[DRONE] VictimAlert — o alerta	4
3.4	[TEAM] VictimAck — a confirmação	4
4	Log estruturado: de texto a tabela consultável	5
4.1	Como o parser resolve o tempo	5
4.2	16 tipos de evento reconhecidos	5
4.3	Do log ao Parquet consultável	5
5	Estudo de caso: por que um alerta não chega	7
5.1	Passo 1 — a tabela de mensagens mostra o quê, não o porquê	7
5.2	Passo 2 — IP diz que a rota está boa	7
5.3	Passo 3 — ARP é a causa raiz real	7
6	Por baixo do capô: rádio, MAC e roteamento AODV	9
6.1	Camada física (PHY) — rádio 802.11	9
6.2	Camada de enlace (MAC) — handshake 802.11	9
6.3	Roteamento AODV — descoberta de rota	9
6.4	ARP — a camada esquecida entre AODV e o rádio	10
7	Timers e parâmetros de controle	11
7.1	Timers internos do drone (SimpleDroneApp)	11
8	Log real anotado — uma simulação inteira em miniatura	12
9	Métricas gravadas ao final (finish())	13
9.1	Do lado do drone	13
9.2	Do lado da equipe	13
10	Glossário rápido	14

1 Por que este guia existe

Este documento explica, item por item, o que aparece nos logs quando a simulação ECHOSAR-Net roda. O objetivo é permitir apresentar e defender cada linha de log para a banca/orientação: o que ela significa, de onde vem no código, e por que aquele valor apareceu ali.

Ideia central da simulação

Drones sobrevoam uma área simulando busca e resgate (SAR). Não há vítimas físicas nem sensores reais de detecção — os drones geram **alertas sintéticos** em intervalos aleatórios para gerar tráfego de rede representativo. O que se mede é a **qualidade da comunicação** entre drones e equipes de resgate em terra: o alerta chega? Em quanto tempo? Quantas retransmissões precisou?

1.1 Como gerar um log legível

Por padrão, a simulação roda em modo “express” (rápido, sem imprimir eventos) para não desperdiçar tempo de CPU. Para ver as mensagens de aplicação ([DRONE], [TEAM]), é preciso pedir o nível INFO explicitamente. `run.sh` oferece duas flags para isso:

```
./run.sh --info -c SmokeTest_Beacons # TUDO: PHY/MAC/AODV/app, muito verboso
./run.sh --msglog -c SmokeTest_Beacons # s troca de mensagens (app), resto em WARN
```

O que as flags fazem de verdade

O script `run.sh` passa duas flags ao motor de simulação: `-cmdenv-log-level` (qual severidade mostrar) e `-cmdenv-express-mode` (se os eventos são impressos ou não). **As duas precisam estar certas** — só pedir `log-level=INFO` não basta se o `express-mode` estiver ligado, porque ele suprime a impressão de qualquer evento independente do nível. `-msglog` desliga o `express-mode` mas mantém o nível global em `WARN`, contando com um override específico do módulo de aplicação (`**app[0].cmdenv-log-level = INFO` no `omnetpp.ini`) para mostrar só as mensagens [DRONE]/[TEAM] — o resultado é ~18% do volume de linhas do `-info` puro, muito mais fácil de ler.

Duas pegadinhas reais de `cmdenv-log-level` — horas de debugging

1. A flag crua vira teto global. Passar `-cmdenv-log-level=WARN` na linha de comando (sem o prefixo `**`.) cria uma entrada de configuração especial que **ignora qualquer override por módulo no ini** — mesmo com `**app[0].cmdenv-log-level=INFO` certo no arquivo, o log fica vazio. É preciso usar `-**.cmdenv-log-level=WARN` (com o prefixo) para que a flag participe da resolução por especificidade normal, como qualquer parâmetro NED — só assim o override mais específico (`app[0]`) consegue vencer o geral.

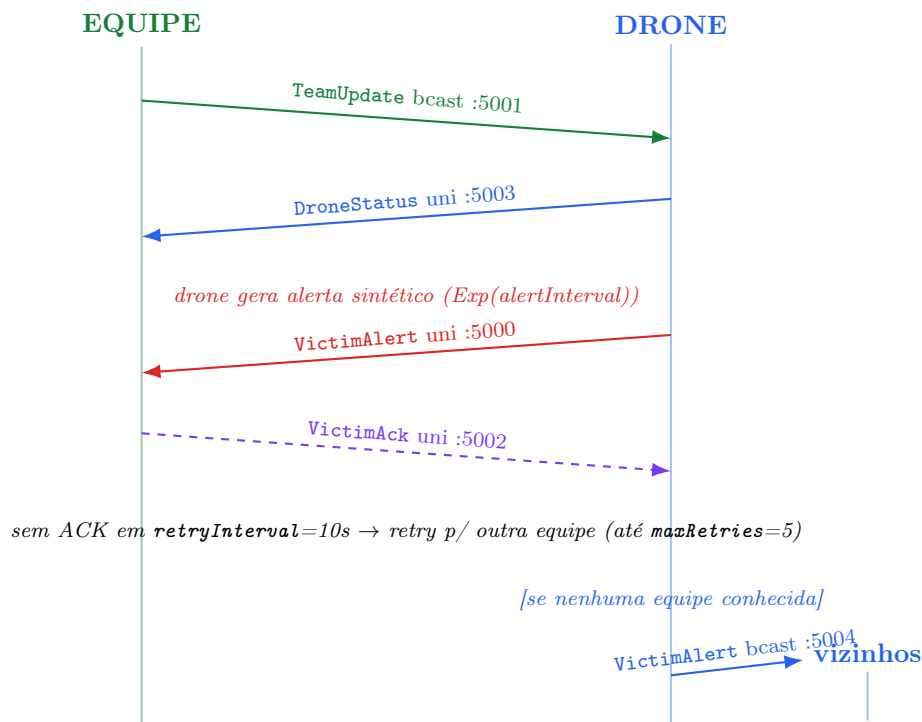
2. O nível não herda de módulo composto para submódulos. Definir `**drone[16].cmdenv-log-level=INFO` não liga o log dos submódulos internos (`app[0]`, `aodv`, `wlan[0].mac`, `wlan[0].radio`...) — esses só logam em `EV_INFO` dentro do próprio código deles, então o padrão precisa casar exatamente com o módulo que *chama* a macro. Para pegar tudo dentro de um nó, é preciso um `**` no meio do padrão: `**drone[16]**.cmdenv-log-level=INFO`.

2 Quem fala com quem: a arquitetura de mensagens

Cinco tipos de mensagem UDP circulam na rede, cada uma numa porta fixa (`src/app/ports.h`):

Porta	Mensagem	Sentido	Quando é enviada
5000	VictimAlert	drone → equipe	Drone gera alerta sintético e escolhe a equipe mais próxima conhecida
5001	TeamUpdate	equipe → broadcast	Beacon periódico da equipe (posição + IP)
5002	VictimAck	equipe → drone origem	Equipe confirma recebimento do alerta
5003	DroneStatus	drone → equipe	Resposta do drone ao receber um TeamUpdate
5004	VictimAlert	drone → drone (relay)	Drone repassa alerta em broadcast quando não conhece nenhuma equipe

2.1 Diagrama de sequência do fluxo completo



Por que o ACK volta direto ao drone que criou o alerta, não a quem repassou

O campo `originIp` dentro do **VictimAlert** guarda o IP do drone que **originou** o alerta. Mesmo que a mensagem tenha passado por vários relays até chegar à equipe, o **VictimAck** é endereçado direto a esse IP de origem via roteamento AODV multi-hop — não ao último drone que fez relay.

3 Anatomia de cada mensagem

3.1 [DRONE] TeamUpdate — beacon da equipe

```
[INFO] [TEAM team0] TeamUpdate broadcast (ip=10.0.0.2)
[INFO] [DRONE drone[0]] tabela: team0 ip=10.0.0.2
```

Campos do TeamUpdateChunk — 128 bytes

teamId (nome da equipe) · **ipAddress** (IP dela na sub-rede 10.0.0.x/24) · **posX/posY** (posição cartesiana no playground, em metros).

O que acontece: a equipe transmite isso em broadcast a cada **sendInterval** segundos (com um jitter aleatório inicial para não colidir com outras equipes no rádio). Todo drone que recebe atualiza sua **teamTable** — é assim que o drone *descobre* quais equipes existem e onde estão, sem nenhuma configuração estática.

3.2 [DRONE] DroneStatus — resposta do drone

```
[INFO] [TEAM team0] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.004653659192s
```

Campos do DroneStatusChunk — 128 bytes

droneId · **posX/posY/posZ** (posição 3D, incluindo altitude) · **sentAt** (timestamp de envio, usado para calcular o RTT mostrado no log).

O que acontece: é a resposta automática do drone a todo **TeamUpdate** recebido — funciona como um “ping” de posição. A equipe usa isso para saber que drones estão por perto e ativos (não é usado para nenhuma métrica de entrega, é diagnóstico de conectividade).

3.3 [DRONE] VictimAlert — o alerta

```
[INFO] [DRONE drone[0]] alerta sintético gerado → drone[0]_1
[INFO] [DRONE drone[0]] VictimAlert drone[0]_1 → team0 (10.0.0.2)
[INFO] [TEAM team0] *** ALERTA de drone[0] msgId=drone[0]_1 vitima em (2500,2500) delay=0.000134368468s
```

Campos do VictimAlertChunk — 256 bytes

droneId · **msgId** (identificador único **droneId_N**, usado para deduplicação) · **originIp** (IP de quem originou, para o ACK voltar certo) · **posX/posY** (posição da “vítima”) · **sentAt**.

O que acontece: a cada intervalo exponencial (**alertInterval**), o drone gera um alerta sintético e escolhe a equipe conhecida **mais próxima** da sua **teamTable** para enviar via unicast. Se não conhece nenhuma equipe, faz relay em broadcast (porta 5004) para drones vizinhos tentarem entregar.

3.4 [TEAM] VictimAck — a confirmação

```
[INFO] [TEAM team0] VictimAck drone[0]_1 → drone origem 10.0.0.1
[INFO] [DRONE drone[0]] VictimAck recebido para drone[0]_1 de team0
```

Campos do VictimAckChunk — 64 bytes

msgId (referencia o alerta confirmado) · **teamId** (quem confirmou) · **sentAt**.

O que acontece: ao receber e processar um **VictimAlert**, a equipe manda esse ACK de volta ao **originIp**. O drone remove o alerta de **pendingAlerts** e para de retransmiti-lo. Se o mesmo **msgId** chegar de novo (ACK perdido no caminho), a equipe reenvia o ACK sem contar de novo nas métricas — evita retransmitir o alerta inteiro por perda só do ACK.

4 Log estruturado: de texto a tabela consultável

Ler milhares de linhas de log na tela não escala para estudar padrões (quantos alertas expiraram? qual a distribuição de retries?). O pipeline `analysis/parse_message_log.py` converte o log de `-msglog/-info` numa tabela em formato longo — 1 linha por evento de troca de mensagem — exportável para CSV ou Parquet e consultável via SQL.

4.1 Como o parser resolve o tempo

A linha de log da aplicação (`[INFO] [DRONE ...] ...`) **não carrega timestamp** — o `simtime` só aparece no cabeçalho de evento do `Cmdenv`, que vem antes:

```
** Event #2511 t=14.715488171128 98% completed (98% total) BasicNetwork.team[1].app[0] (SimpleTeamApp, id=186)
[INFO] [TEAM team1] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.000204403078s
```

Por que isso funciona sem ambiguidade

O simulador é orientado a eventos discretos e **single-threaded** — cada `** Event #N` é processado até o fim antes do próximo começar. O parser varre o arquivo linha a linha, guarda o `t=` mais recente visto, e o aplica a toda linha `[INFO]/[WARN]` seguinte até o próximo cabeçalho de evento — sem risco de misturar tempos de eventos diferentes.

4.2 16 tipos de evento reconhecidos

Cada linha `[DRONE ...]/[TEAM ...]` bate contra uma lista de expressões regulares (uma por formato de mensagem do código-fonte) e vira uma linha da tabela com colunas: `time`, `event`, `drone_id`, `team_id`, `msg_id`, `peer_id`, `ip`, `x`, `y`, `z`, `delay`, `retries`, `max_retries` — nulas quando não se aplicam ao evento.

Evento	Origem	Quando ocorre
<code>team_table_update</code>	drone	atualiza entrada da <code>teamTable</code> ao receber <code>TeamUpdate</code>
<code>alert_generated</code>	drone	alerta sintético criado
<code>alert_sent_direct</code>	drone	<code>VictimAlert</code> unicast a uma equipe conhecida
<code>alert_sent_relay</code>	drone	<code>VictimAlert</code> broadcast (sem equipe elegível)
<code>alert_dedup_drone</code>	drone	descarta relay já visto (<code>seenAlerts</code>)
<code>alert_relay_received</code>	drone	recebeu relay de outro drone e vai repassar
<code>ack_received</code>	drone	<code>VictimAck</code> recebido, remove de <code>pendingAlerts</code>
<code>alert_expired</code>	drone	descartado após <code>maxRetries</code> sem ACK
<code>retry</code>	drone	tentativa de store-forward
<code>team_timeout</code>	drone	remove equipe inativa da <code>teamTable</code>
<code>team_update_sent</code>	equipe	beacon <code>TeamUpdate</code> enviado
<code>drone_status_received</code>	equipe	<code>DroneStatus</code> recebido, RTT calculado
<code>alert_dedup_team</code>	equipe	reenvia ACK idempotente p/ alerta já visto
<code>alert_received</code>	equipe	*** ALERTA — entrega confirmada
<code>ack_sent</code>	equipe	<code>VictimAck</code> enviado ao <code>originIp</code>
<code>unexpected_packet</code>	equipe	pacote não reconhecido no socket

4.3 Do log ao Parquet consultável

```
# roda a sim com --msglog, parseia e converte para parquet numa chamada
analysis/export_messages_parquet.sh BasicTest_Piloto 0
```

```
# ou manualmente:
./run.sh --msglog -c BasicTest_Piloto -r 0 > /tmp/run.log
python3 analysis/parse_message_log.py /tmp/run.log > /tmp/messages.csv
```

Por que Parquet e não só CSV

O CSV já é útil, mas o Parquet é columnar e indexável — consultas por `msg_id` ou `event` em dezenas de milhares de linhas rodam em milissegundos via `duckdb`, sem precisar carregar tudo em memória como um `DataFrame` pandas. E o formato é o mesmo já usado por `analysis/export_parquet.sh` para os `.sca/.vec` — um único jeito de consultar métricas agregadas e eventos linha-a-linha lado a lado.

```
# reconstroi o ciclo completo de um alerta especifico
duckdb -c "SELECT * FROM read_parquet('analysis/parquet/BasicTest_Piloto-0_messages.
      parquet')
      WHERE msg_id='drone[0]_1' ORDER BY time"

# distribuicao de eventos (visao geral do que aconteceu na simulacao)
duckdb -c "SELECT event, count(*) FROM read_parquet('analysis/parquet/BasicTest_Piloto-0
      _messages.parquet')
      GROUP BY event ORDER BY 2 DESC"

# alertas gerados antes do drone conhecer qualquer equipe (candidatos a falha por
      isolamento)
duckdb -c "
WITH first_team AS (
  SELECT drone_id, min(time) AS first_seen
  FROM read_parquet('analysis/parquet/BasicTest_Piloto-0_messages.parquet')
  WHERE event='team_table_update' GROUP BY drone_id)
SELECT m.msg_id, m.time AS generated_at, f.first_seen
FROM read_parquet('analysis/parquet/BasicTest_Piloto-0_messages.parquet') m
JOIN first_team f USING (drone_id)
WHERE m.event='alert_generated' AND m.time < f.first_seen
ORDER BY m.time"
```

5 Estudo de caso: por que um alerta não chega

Esta seção documenta uma investigação real feita neste projeto — `drone[6]_1` gerado em `BasicTest_Piloto` (seed 0) expirou após 5 retries sem nunca ser confirmado, mesmo tendo sido **relayado com sucesso** até um drone que conhecia a equipe destino. O caminho até a causa raiz mostra como cruzar os três níveis de log (app, tabela de Parquet, PHY/MAC/ARP bruto).

5.1 Passo 1 — a tabela de mensagens mostra o quê, não o porquê

```
t=6.55028635543 [DRONE drone[6]] alerta sintetico gerado -> drone[6]_1
t=6.55028635543 [DRONE drone[6]] relay: drone[6]_1 -> broadcast (sem equipe elegivel)
t=6.551187119027 [DRONE drone[16]] relay recebido: drone[6]_1 de drone[6]
t=6.551187119027 [DRONE drone[16]] VictimAlert drone[6]_1 -> team0 (10.0.0.21)
t=10.0 [DRONE drone[6]] store-forward: retry 1/5 para drone[6]_1
... (mais 4 retries, um por 10s) ...
t=60.0 [DRONE drone[6]] store-forward: descartando drone[6]_1 apos 5 tentativas
```

O que a tabela de eventos já revela

O drone origem (`drone[6]`) não conhecia nenhuma equipe, então fez relay em broadcast. Um vizinho (`drone[16]`) recebeu o relay, **conhecia** `team0` e reenviou o `VictimAlert` unicast para lá — o mecanismo de relay funcionou como projetado. Mas `team0` nunca gerou o log ***** ALERTA de drone[6]**: o pacote se perdeu em algum lugar entre `drone[16]` e `team0`, mesmo sendo vizinhos diretos. A tabela de eventos de aplicação não diz onde — para isso, é preciso descer de camada.

5.2 Passo 2 — IP diz que a rota está boa

```
t=6.551187119027 drone[16].ipv4.ip Finding route for source <unspec> with destination 10.0.0.21
t=6.551187119027 drone[16].ipv4.ip Active route found: destination = 10.0.0.21,
prefixLength = 32, nextHop = 10.0.0.21, metric = 1, interface = wlan0
```

O AODV está satisfeito: rota ativa, 1 salto, próximo salto é o próprio destino. Se a investigação parasse aqui, pareceria bug de roteamento — mas a rota AODV é só metade da história.

5.3 Passo 3 — ARP é a causa raiz real

```
t=1.506 drone[16].ipv4.arp ARP request for 10.0.0.21 timed out, resending
t=2.506 drone[16].ipv4.arp ARP request for 10.0.0.21 timed out, resending
t=3.506 drone[16].ipv4.arp ARP timeout, max retry count 3 for 10.0.0.21 reached.
t=3.506 drone[16].ipv4.arp ARP resolution failed for 10.0.0.21, dropping 7 packets
t=5.447 drone[16].ipv4.arp ARP request for 10.0.0.21 timed out, resending
t=6.447 drone[16].ipv4.arp ARP request for 10.0.0.21 timed out, resending
t=7.447 drone[16].ipv4.arp ARP timeout, max retry count 3 for 10.0.0.21 reached.
t=7.447 drone[16].ipv4.arp ARP resolution failed for 10.0.0.21, dropping 9 packets
```

O gap entre AODV e ARP

AODV resolve rota **IP → próximo salto**; ARP resolve **IP → endereço MAC** — são caches independentes. A rota AODV para `team0` já existia (RREP recebido antes), mas o cache ARP de `drone[16]` para esse mesmo IP nunca foi preenchido: o *ARP request* sai em broadcast, tenta 3 vezes (retry a cada 1s), e `team0` nunca responde — provável colisão do broadcast num instante de contenção de canal alta (rede densa, muitos nós tentando transmitir). Quando o timeout final bate, **todos os pacotes enfileirados para aquele destino são descartados de uma vez** — `VictimAlert drone[6]_1` estava entre os 9 descartados em `t=7.447`, apesar do AODV achar a rota perfeitamente boa. Uma rota ativa não garante ARP resolvido — são duas camadas que podem divergir.

Como reproduzir esta investigação

1. Rode com `-msglog`, exporte para parquet, ache um `msg_id` com `event='alert_expired'`.
2. Consulte esse `msg_id` no parquet (`ORDER BY time`) para ver a cadeia de retries e quem tentou relayar.
3. Identifique o par (drone que tentou enviar, IP de destino) e o instante aproximado da falha.
4. Rode de novo com `-info` completo, mas com `-sim-time-limit` curto (poucos segundos ao redor do instante de interesse) para manter o log gerenciável, e `grep` pelos módulos `ipv4.ip/ipv4.arp` desse par de nós.

6 Por baixo do capô: rádio, MAC e roteamento AODV

Tudo que foi mostrado até aqui ([DRONE]/[TEAM]) é log de **aplicação**. Mas cada uma dessas mensagens desce por várias camadas de rede simuladas antes de virar sinal de rádio — e o **-info** imprime o que acontece em *todas* elas. Esta seção explica as linhas mais frequentes desses níveis mais baixos.

6.1 Camada física (PHY) — rádio 802.11

```
[INFO] Radio mode changed from TRANSMITTER to RECEIVER.
[INFO] Changing radio reception state from UNDEFINED to IDLE.
[INFO] Changing radio transmission state from IDLE to UNDEFINED.
[INFO] bpsk snr=6610.06 ber=0
[INFO] 16-Qam snr=6610.06 ber=6.97908e-290
[INFO] Reception ended: successfully for (...)WlanAck (34 us 27 B) ...
      power = 661.006 pW, centerFrequency = 2.412 GHz, bandwidth = 20 MHz
[INFO] Sending up (inet::Packet)WlanAck (14 B)
[INFO] Changing radio reception state from RECEIVING to IDLE.
```

O que cada linha quer dizer

Radio mode / reception state / transmission state: máquina de estados do rádio (IDLE, RECEIVING, TRANSMITTER...) — um rádio half-duplex só pode estar em um estado por vez.
bpsk/16-Qam snr=... ber=...: para *cada* modulação suportada, o simulador calcula a **SNR** (relação sinal-ruído, linear) e a **BER** (taxa de erro de bit) que aquele link teria se usasse essa modulação — serve para decidir se o quadro é decodificado com sucesso.
power = ... pW: potência recebida do sinal, em picowatts — compare com a sensibilidade do receptor (-85 dBm no cenário) para saber se o quadro sequer é detectável.

6.2 Camada de enlace (MAC) — handshake 802.11

```
[INFO] Received frame from PHY: (inet::Packet)WlanAck (14 B)
[INFO] Processing lower frame: WlanAck
[INFO] Finishing last frame sequence step: history = ({ } LAST-FRAME ACK)
[INFO] Processing received frame WlanAck as originator in frame sequence.
[INFO] Frame sequence finished.
[INFO] Channel released.
```

Por que todo pacote de aplicação vira um WlanAck

O IEEE 802.11 exige confirmação em nível de enlace para **todo** quadro unicast: quem envia um DATA espera um ACK do MAC antes de liberar o canal (**Channel released**). Isso é diferente do **VictimAck** da aplicação — este ACK é interno ao rádio, acontece em microssegundos, e existe até para um simples **TeamUpdate**. Se o ACK 802.11 não chegar, o MAC retransmite no nível de enlace, *antes* de qualquer retry da aplicação entrar em ação.

6.3 Roteamento AODV — descoberta de rota

Quando um drone precisa mandar algo para um IP com o qual ainda não tem rota ativa, o AODV descobre a rota sob demanda. Exemplo real de 10.0.0.1 (drone) descobrindo rota para 10.0.0.2 (equipe) para poder enviar um **DroneStatus**:

```
[INFO] Finding route for source <unspec> with destination 10.0.0.2
[INFO] Missing route for destination 10.0.0.2
[INFO] Starting route discovery with originator 10.0.0.1 and destination 10.0.0.2
[INFO] Sending a Route Request with target 10.0.0.2 and TTL= 2
...
[INFO] AODV Route Request arrived with source addr: 10.0.0.1 originator addr: 10.0.0.1 destination addr:
      10.0.0.2
[INFO] I am the destination node for which the route was requested
[INFO] Sending Route Reply to 10.0.0.1
```

```
...
[INFO] AODV Route Reply arrived with source addr: 10.0.0.2 originator addr: 10.0.0.1 destination addr:
      10.0.0.2
[INFO] The Route Reply has arrived for our Route Request to node 10.0.0.2
[INFO] Routing (inet::Packet)DroneStatus (156 B) with destination = 10.0.0.2,
      output interface = wlan0, next hop address = 10.0.0.2
```

Leitura linha a linha

1. **Missing route** — o drone não tem entrada na tabela de roteamento para 10.0.0.2, então dispara descoberta.
2. **RREQ** (*Route Request*) sai em broadcast com TTL=2 — alcança até 2 saltos.
3. Quem recebe o RREQ e é o destino responde direto com **RREP** (*Route Reply*); quem não é, encaminharia o RREQ adiante (aqui não precisou, são vizinhos diretos).
4. Ao receber o RREP, o drone grava a rota (**add route**) e agora pode **rotear** o DroneStatus de verdade, com **next hop** definido.

Isso só acontece **uma vez** por par origem/destino (enquanto a rota estiver ativa — **activeRouteTimeout=5s**). Depois disso, os pacotes seguintes só aparecem como **Active route found**, sem todo o handshake RREQ/RREP de novo.

6.4 ARP — a camada esquecida entre AODV e o rádio

Ter uma rota AODV ativa não é suficiente para transmitir: o quadro 802.11 precisa de um endereço **MAC** de destino, e quem resolve IP → MAC é o ARP — uma camada totalmente separada, com seu próprio cache e timeout.

```
[INFO] Received (inet::Packet)arpREQ (28 B) from network protocol.
[INFO] IPv4 address 10.0.0.11 not recognized, dropping ARP packet
...
[INFO] ARP resolution completed for 10.0.0.11. Sending 1 waiting packets from the queue
...
[INFO] ARP request for 10.0.0.21 timed out, resending
[INFO] ARP timeout, max retry count 3 for 10.0.0.21 reached.
[INFO] ARP resolution failed for 10.0.0.21, dropping 7 packets
```

Leitura linha a linha

ARP request/reply: quando um nó precisa mandar algo a um IP sem MAC resolvido, ele transmite um *ARP request* em broadcast (“quem tem esse IP?”); o dono responde com *ARP reply* unicast.

“not recognized, dropping”: um nó recebeu um ARP request/reply que não é destinado a ele (broadcast alcança todo mundo no alcance de rádio) — comportamento normal, não é erro.

“resolution completed... Sending N waiting packets”: enquanto o ARP não resolve, os pacotes de dados para aquele destino ficam **enfileirados** (não descartados ainda) — assim que a resposta chega, a fila inteira é liberada de uma vez.

“timed out, resending” / **“max retry count”**: se o ARP reply não chega, o request é reenviado (até 3 vezes neste cenário); esgotadas as tentativas, **toda a fila daquele destino é descartada** — este é o mecanismo por trás do estudo de caso da Seção 5.

7 Timers e parâmetros de controle

Parâmetro	Onde	Papel
<code>alertInterval</code>	drone (.ned)	Média da distribuição exponencial entre alertas sintéticos gerados
<code>teamTimeout</code>	drone (.ned)	Se uma equipe some da <code>teamTable</code> por esse tempo sem novo <code>TeamUpdate</code> , é removida
<code>retryInterval</code>	drone (.ned)	Intervalo entre tentativas de reenvio de um alerta sem ACK
<code>maxRetries</code>	drone (.ned)	Máximo de retransmissões antes do alerta ser descartado (<code>alertsExpired</code>)
<code>sendInterval</code>	equipe (.ned)	Intervalo entre beacons <code>TeamUpdate</code>
<code>beaconJitter</code>	equipe (.ned)	Atraso aleatório do primeiro beacon — evita colisão MAC entre equipes

Por que o `beaconJitter` é crítico

Se todas as equipes transmitirem o primeiro beacon exatamente no mesmo instante, o rádio 802.11 sofre colisões simultâneas e **nenhum** drone recebe `TeamUpdate` — a `teamTable` fica vazia e o PDR despenca para perto de zero. O jitter espalha as transmissões iniciais no tempo. O código rejeita (`cRuntimeError`) se `beaconJitter=0` exatamente por esse motivo já ter causado uma regressão real na simulação.

7.1 Timers internos do drone (SimpleDroneApp)

- `alertTimer` — dispara `generateAlert()` a cada `Exp(alertInterval)`
- `timeoutTimer` — dispara `checkTimeouts()` a cada `teamTimeout`, limpando equipes inativas
- `retryTimer` — dispara `retryPending()` a cada `retryInterval`, reenviando alertas sem ACK

8 Log real anotado — uma simulação inteira em miniatura

Trecho real de `SmokeTest_Beacons` (1 drone parado + 2 equipes, $t = 0$ a 15s) rodado com `./run.sh -info -c SmokeTest_Beacons`. Cada bloco abaixo é um ciclo completo do protocolo:

```
[INFO] [TEAM team0] TeamUpdate broadcast (ip=10.0.0.2)
[INFO] [DRONE drone[0]] tabela: team0 ip=10.0.0.2
[INFO] [TEAM team0] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.004653659192s
[INFO] [TEAM team1] TeamUpdate broadcast (ip=10.0.0.3)
[INFO] [DRONE drone[0]] tabela: team1 ip=10.0.0.3
[INFO] [TEAM team1] DroneStatus de drone[0] pos=(2500,2500,100) RTT=0.001449209234s
[INFO] [DRONE drone[0]] alerta sintético gerado → drone[0]_1
[INFO] [DRONE drone[0]] VictimAlert drone[0]_1 → team0 (10.0.0.2)
[INFO] [TEAM team0] *** ALERTA de drone[0] msgId=drone[0]_1 vitima em (2500,2500) delay=0.000134368468s
[INFO] [TEAM team0] VictimAck drone[0]_1 → drone origem 10.0.0.1
[INFO] [DRONE drone[0]] VictimAck recebido para drone[0]_1 de team0
```

Leitura linha a linha

1. As duas equipes anunciam sua posição/IP (`TeamUpdate`) em momentos diferentes — efeito do `beaconJitter`.
2. O drone confirma a recepção de cada beacon com `DroneStatus` — por isso aparecem dois RTTs, um por equipe.
3. O drone gera um alerta sintético (`drone[0]_1`) e escolhe `team0` — nesta simulação, a única candidata elegível no momento (equipe mais próxima entre as conhecidas).
4. A equipe recebe, imprime `*** ALERTA ***` (marcador visual de entrega bem-sucedida) e calcula o `delay` — tempo entre `sentAt` do alerta e o instante de recepção. Aqui, 0,13 ms: drone e equipe estão a poucos metros um do outro no cenário de teste.
5. A equipe devolve o `VictimAck` ao IP de origem do drone.
6. O drone recebe o ACK e remove o alerta de `pendingAlerts` — ciclo fechado, sem retransmissão necessária.

9 Métricas gravadas ao final (`finish()`)

Ao fim de cada simulação, cada módulo grava contadores (`scalar`) no arquivo `.sca` — é isso que `analysis/process_results.py` lê para calcular as métricas da dissertação.

9.1 Do lado do drone

Scalar	Tipo	Significado
<code>alertsGenerated</code>	contador	alertas sintéticos criados
<code>alertsSentDirect</code>	contador	<code>VictimAlert</code> enviados direto a uma equipe
<code>alertsSentRelay</code>	contador	<code>VictimAlert</code> enviados em relay broadcast
<code>alertsRelayed</code>	contador	alertas de <i>outro</i> drone recebidos e repassados
<code>alertsAcked</code>	contador	<code>VictimAck</code> recebidos (ciclo completo confirmado)
<code>alertsExpired</code>	contador	descartados após <code>maxRetries</code> sem ACK
<code>totalRetries</code>	contador	soma de tentativas de store-forward
<code>meanRTT</code>	média	tempo alerta→ACK, do lado do drone (ciclo completo, não é o atraso 1-via)

9.2 Do lado da equipe

Scalar	Tipo	Significado
<code>alertsReceived</code>	contador	<code>VictimAlert</code> únicos recebidos (após deduplicação)
<code>teamUpdatesSent</code>	contador	beacons enviados
<code>droneStatusReceived</code>	contador	respostas de drones recebidas
<code>meanDeliveryDelay</code>	média	atraso de entrega 1-via (drone→equipe) — esta é a métrica de “atraso fim-a-fim” reportada na dissertação

PDR vs. AppACK — por que existem duas métricas de sucesso

PDR = `alertsReceived/alertsGenerated` mede *chegada ao destino* (a equipe recebeu). **AppACK** = `alertsAcked/alertsGenerated` mede o *ciclo completo* confirmado (o drone soube que chegou). Com AODV multi-hop os dois quase coincidem, porque o `VictimAck` também consegue voltar vários saltos — a diferença residual vem de ACKs perdidos no caminho de volta.

10 Glossário rápido

Termo	Significado
AODV	Protocolo de roteamento reativo multi-hop — descobre rotas sob demanda (RREQ/RREP), sem tabelas estáticas
RREQ / RREP	<i>Route Request</i> / <i>Route Reply</i> — mensagens do AODV para descobrir e confirmar uma rota
SNR / BER	Relação sinal-ruído / taxa de erro de bit — determinam se um quadro de rádio é decodificado com sucesso
ARP	Protocolo que resolve IP → endereço MAC antes de enviar um quadro no enlace — cache e timeout independentes da rota AODV; pode falhar mesmo com rota AODV ativa
FANET	Flying Ad-hoc Network — rede ad hoc formada pelos próprios drones em voo
-msglog	Flag do <code>run.sh</code> : log INFO só da aplicação (<code>[DRONE]</code> / <code>[TEAM]</code>), resto em WARN — usa <code>parse_message_log.py</code> para virar tabela
Parquet	Formato de arquivo columnar; aqui guarda tanto métricas agregadas (<code>.sca/.vec</code> via <code>export_parquet.sh</code>) quanto eventos de mensagem linha-a-linha (via <code>export_messages_parquet.sh</code>)
DuckDB	Motor SQL embutido usado para consultar os arquivos Parquet direto do terminal, sem servidor de banco
Relay	Repasse de um alerta por um drone que não tem equipe conhecida, esperando que um vizinho tenha
Store-forward	Estratégia de reter o alerta e reenviar periodicamente até receber ACK ou esgotar tentativas
Dedup / <code>seenAlerts</code>	Conjunto de <code>msgId</code> já vistos, evita processar/retransmitir o mesmo alerta duas vezes
Express-mode	Modo do <code>Cmdenv</code> que roda sem imprimir eventos (rápido); precisa ser desligado para ver logs
<code>EV_INFO</code> / <code>EV_WARN</code>	Macros do OMNeT++ para logging em diferentes níveis de severidade
Scalar (<code>.sca</code>)	Arquivo com os contadores finais de cada módulo, um por seed
Seed / <code>run</code>	Uma execução independente da simulação com número aleatório diferente — várias seeds dão média $\pm$ desvio

Gerado a partir do código em `src/app/{SimpleDroneApp,SimpleTeamApp}.{h,cc,ned}`, de execuções reais de `SmokeTest_Beacons` e `BasicTest_Piloto`, e do pipeline `analysis/parse_message_log.py` — ver `CLAUDE.md` para detalhes de arquitetura.